

d)

```

public static void rotacionar (int [] V, int tam, int k) {
    if (tam <= 1) {
        return;
    }

```

$k = k \% tam;$

```

    if (k > 0) {
        k = k + tam;
    }

```

```

    inverte (V, 0, k-1);
    inverte (V, k, tam-1);
    inverte (V, 0, tam-1);
}

```

Função auxiliar

```

public static void inverte (int [] V, int inicio inicio, int fim) {
    while (inicio < fim) {
        int aux = V[inicio];
        V[inicio] = V[fim];
        V[fim] = aux;

```

$inicio++;$

$fim--;$

```

    }
}

```

Tempo para fazer as 4 questões: 50 ~~min~~ minutos

data 8 / 10 / 2019

b)

```

public static void ordenar (int [] v, int m) {
    for (int i = 1; i < m; i++) {
        int chave = v[i];
        int j = i - 1;

        while (j >= 0 && v[j] > chave) {
            v[j+1] = v[j];
            j--;
        }

        v[j+1] = chave;
    }
}

```

c)

```

public static int gerarVetorSemRepeticao (int [] v, int tamV, int [] vnr) {
    int tamVnr = 0;

    for (int i = 0; i < tamV; i++) {
        if (!pertence (vnr, tamVnr, v[i])) {
            vnr[tamVnr] = v[i];
            tamVnr++;
        }
    }

    return tamVnr;
}

```

a)

```
public static int uniao (int[] a, int tamA, int[] b, int tamB, int[] u) {
    int tamU = 0;
```

```
    for (int i = 0; i < tamA; i++) {
        if (!pertence (u, tamU, a[i])) {
            u[tamU] = a[i];
            tamU++;
        }
    }
```

```
    for (int i = 0; i < tamB; i++) {
        if (!pertence (u, tamU, b[i])) {
            u[tamU] = b[i];
            tamU++;
        }
    }
```

```
    return tamU;
}
```

Função auxiliar

```
public static bool boolean pertence (int[] v, int tam, int valor) {
    for (int i = 0; i < tam; i++) {
        if (v[i] == valor) {
            return true;
        }
    }
    return false;
}
```